

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..20

WORKING WITH USERS

 Copy as Markdown

Working with users

The authentication module provides convenient ways to work with your authenticated users and their related profile data.

Authenticated users

All authenticated users have an authentication identifier that uniquely identifies them across the server. This can be retrieved from the `session` object as a `String` through the `userIdentifier` property or as a `UuidValue` from the `authUserId` extension provided by the authentication module.

```
var userIdString = session.authenticated?.userIdentifier;
// requires `import 'package:serverpod_auth_idp_server/core.dart';`
var userIdUuidValue = session.authenticated?.authUserId;
```

Further operations on the authenticated user can be performed using the `AuthUsers` class which is provided by the `AuthServices` instance.

```
await AuthServices.instance.authUsers.delete(session, authUserId: userIdUuidValue!);
```

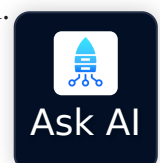
For the full list of operations, see the [AuthUsers](#) class documentation.

Blocking users

You can block users to prevent them from signing in to your app. When a blocked user attempts to authenticate, an `AuthUserBlockedException` will be thrown, and the authentication will fail.

Blocking or unblocking a user

To block/unblock a user, use the `update` method of the `AuthUsers` class:



```
await AuthService.instance.authUsers.update(  
  session,  
  authUserId: authUserId,  
  blocked: true, // or false to unblock  
);
```

Users can also be created with the blocked status set from the start:

```
await AuthService.instance.authUsers.create(  
  session,  
  blocked: true,  
);
```

NOTE

When a user is blocked, they will not be able to sign in until they are unblocked. However, blocking a user does not automatically revoke their existing tokens. Be sure to revoke existing sessions for a complete block operation. See [Revoking tokens](#) for more details.

User creation callbacks

You can react when an auth user is created and control their initial scopes and blocked status by using the `AuthUsersConfig` callbacks. Configure them when initializing auth services on the `pod` object.

WARNING

Both callbacks receive a `transaction` parameter that should be used on all operations performed inside the callback. Failing to pass the transaction to database operations might lead to entries not being found or changes not being rolled back together.

Reacting to the user created event

Use the `onAfterAuthUserCreated` callback to run logic after a new auth user has been created (for example, to create related domain data or send a welcome notification). The callback receives the current session, the newly created auth user, and the ongoing transaction.

```
pod.initializeAuthServices(  
  ...  
  authUsersConfig: AuthUsersConfig(  
    onAfterAuthUserCreated: (session, authUser, {required transaction}) {  
      // Do something with the new auth user (e.g. create related data)  
    },  
  ),  
);
```

```
);
```

Setting default scopes and blocked status

Use the `onBeforeAuthUserCreated` callback to set default scopes or blocked status for new auth users. The callback receives the session, the scopes and blocked value that would be used by default, and the transaction. Return a record with the `scopes` and `blocked` values you want to apply. You can add or remove scopes, or force the user to be blocked.

```
pod.initializeAuthServices(  
  ...  
  authUsersConfig: AuthUsersConfig(  
    onBeforeAuthUserCreated: (session, scopes, blocked, {required transaction}) {  
      // Set default scopes (e.g. add Scope.admin) and optionally block the user  
      return (scopes: {...scopes, Scope.admin}, blocked: blocked);  
    },  
  ),  
);
```

User profiles

Authenticated users get a profile with information about the signed-in user. Read it with the `userProfile` extension on `AuthenticationInfo`. The extension returns a `UserProfileModel`, which is the read model sent to the app. The `UserProfile` class itself is only used for database access.

```
var userProfile = await session.authenticated?.userProfile(session);
```

The result is `null` only when no user is signed in. If the user is signed in but has no profile, the call throws `UserProfileNotFoundException`.

The profile contains a basic set of information about the user, such as their full name, email address, and profile picture.

This information is automatically populated when the user signs in. Based on the authentication method used, different data may be available.

It's a common task to read or update user information on your server. The `UserProfiles` class provides many convenient methods for working with user profiles and is accessible through the `AuthServices` instance.

```
await AuthServices.instance.userProfiles.changeFullName(session, authUserId, 'my name');
```

For the full list of operations, see the [UserProfiles](#) class documentation.

User profile callbacks

You can react when a user profile is created or updated by using the `UserProfileConfig` callbacks. Configure them when initializing auth services on the `pod` object via the `userProfileConfig` parameter.

⚠ WARNING

All callbacks receive a `transaction` parameter that should be used on all database operations performed inside the callback. Failing to pass the transaction might lead to entries not being found or changes not being rolled back together.

Callback	When	Purpose
<code>onBeforeUserProfileCreated</code>	Before a profile is created	Receive and return <code>UserProfileData</code> to modify defaults (user name, full name, email).
<code>onAfterUserProfileCreated</code>	After a profile is created	Run logic that depends on the new profile (e.g. related data, default image).
<code>onBeforeUserProfileUpdated</code>	Before a profile is updated (name, full name, or picture)	Receive and return <code>UserProfileData</code> to enforce rules or transform values.
<code>onAfterUserProfileUpdated</code>	After a profile is updated	Sync related data or trigger side effects.

Example using `onAfterUserProfileCreated`:

```
pod.initializeAuthServices(  
  ...  
  userProfileConfig: UserProfileConfig(  
    onAfterUserProfileCreated: (session, userProfile, {required transaction}) async {  
      // Do something with the new profile (e.g. create related data, set default image)  
    },  
  ),  
);
```

Accessing user profiles from the app

To access the user profile from your Flutter app, you can use the `userProfileInfo` endpoint that is included in the authentication module:

```
final userProfile = await client.modules.serverpod_auth_core.userProfileInfo.get();
```

This returns a `UserProfileModel` object containing the signed-in user's profile information such as their name, email, and profile picture.

Extending the user profile edit endpoint

The authentication module provides a `UserProfileEditBaseEndpoint` abstract class that you can extend to expose user profile editing functionality to your app. This base endpoint includes methods for:

- Removing user images
- Setting user images
- Changing user names
- Changing full names

For a step-by-step guide to profile photo upload, display, and storage, see [Manage user profile photos](#).

To enable profile editing in your app, create a concrete endpoint class on your server by extending `UserProfileEditBaseEndpoint`:

```
import 'package:serverpod/serverpod.dart';
import 'package:serverpod_auth_idp_server/core.dart';

class UserProfileEditEndpoint extends UserProfileEditBaseEndpoint {}
```

This endpoint will have both `get` (same as `userProfileInfo`) and all profile editing methods. You can then call these methods from your Flutter app:

```
// Get the user's profile
final profile = await client.userProfileEdit.get();

// Change the user's full name
final updatedProfile = await client.userProfileEdit.changeFullName('John Doe');

// Change the user's username
final updatedProfile = await client.userProfileEdit.changeUserName('johndoe');

// Remove the user's profile image
final updatedProfile = await client.userProfileEdit.removeUserImage();

// Set a new profile image
final ByteData imageData = // ... load image data
final updatedProfile = await client.userProfileEdit.setUserImage(imageData);
```

NOTE

The `UserProfileEditBaseEndpoint` requires authentication and operates on the currently authenticated user's profile.

You can also extend the endpoint class to add custom profile editing functionality:

```
class UserProfileEditEndpoint extends UserProfileEditBaseEndpoint {
  Future<UserProfileModel?> myCustomProfileEdit(Session session, String bio) async {
    final userProfile = await session.authenticated?.userProfile(session);

    // Your custom logic here...

    return userProfile;
  }
}
```

Setting a default user image

Some providers, such as Google, expose the user's picture. In that case it is fetched and set as the profile picture automatically. When no picture is available, you can generate a default image with the `onAfterUserProfileCreated` callback in `UserProfileConfig` (see [User profile callbacks](#) for the full set). For the configuration example and the image settings, see [Profile photos](#).

Attaching additional information

The recommended way to attach additional information to an authenticated user is to use a relation in the database. This makes it easy to query the data later based on the user's authentication identifier.

```
class: MyDomainData
table: my_domain_data
fields:
  ### The [AuthUser] this profile belongs to
  authUser: module:serverpod_auth_core:AuthUser?, relation(onDelete=Cascade)
  additionalInfo: String

indexes:
  auth_user_id_unique_idx:
    fields: authUserId
    unique: true
```

The model above creates a relation to the `AuthUser` table and ensures that each user can only have one `MyDomainData` object. The `onDelete=Cascade` ensures that when the `AuthUser` is deleted, the `MyDomainData` object is also deleted.

 **NOTE**

The `AuthUser` model is declared in the `serverpod_auth_core` module, which is automatically included in your project as a dependency of the `serverpod_auth_idp` module. If you are not ignoring the generated files in your `analysis_options.yaml`, you might need to explicitly add the `serverpod_auth_core` module to your project to prevent `depend_on_referenced_packages` lint errors. The general recommendation, however, is to ignore linting on generated files:

```
# analysis_options.yaml
analyzer:
  exclude:
    - lib/src/generated/**
```

TIP

When referencing module classes in your model files, you can use a nickname for the module instead of the full module name. See the [modules documentation](#) for more information.

Query the additional information with the user's `authUserId`:

```
final authUserId = session.authenticated?.authUserId;
final additionalInfo = await MyDomainData.db.findFirstRow(
  session,
  where: (t) => t.authUserId.equals(authUserId!),
);
```

Merging accounts

When a user adds a sign-in method that already belongs to a different account, the two accounts have to become one. Serverpod does not merge them on its own: you decide when to offer the merge, and run it once the user accepts.

A merge runs as an ordered list of hooks (declared in `mergeHooks`). All hooks run inside one transaction, so a failure at any step leaves both accounts untouched. By default, `AccountMergeConfig` fills the list with these hooks:

Hook	What it does
<code>defaultIdpMergeHandler</code>	Calls <code>mergeAuthUsers</code> on every initialized identity provider (e.g. Email, Google, Apple, etc.).
<code>defaultCoreDataMergeHa</code>	Merges the scopes and the blocked flag of the <code>AuthUser</code> records, moves the refresh tokens and server-side sessions, and merges the user profiles. A user blocked on

Hook	What it does
<code>ndler</code>	either account stays blocked after the merge.
<code>applicationMergeHandler</code>	Moves your application's own data. This is the one you write.
<code>defaultMergeCleanupHandler</code>	Deletes the <code>AuthUser</code> that was merged away.

Configure the merge

Your application's own data is the part Serverpod cannot move for you. Pass the

`applicationMergeHandler` to `pod.initializeAuthServices()` that reassigns your rows from the removed user to the kept one:

```
pod.initializeAuthServices(
  tokenManagerBuilders: [...],
  accountMergeConfig: AccountMergeConfig(
    applicationMergeHandler:
      (
        Session session, {
          required UuidValue userToKeepId,
          required UuidValue userToRemoveId,
          required Transaction transaction,
        }) async {
        await MyDomainData.db.updateWhere(
          session,
          where: (t) => t.authUserId.equals(userToRemoveId),
          columnValues: (t) => [t.authUserId(userToKeepId)],
          transaction: transaction,
        );
      },
    ),
  );
```

Your handler only needs to move data over. The `defaultMergeCleanupHandler` runs after it and deletes the removed `AuthUser`, which cascades to all rows that reference it with `onDelete=Cascade`.

Without a handler, merging throws. That is the default for applications that never merge accounts.

To reorder the built-in hooks or replace one of them, use `AccountMergeConfig.custom` and pass the full list yourself.

Merge two users

```
await AuthServices.instance.accountMerger.merge(  
  session,  
  userToKeepId: userToKeepId,  
  userToRemoveId: userToRemoveId,  
);
```

Both users must exist and be different from each other, otherwise the call throws.

Related

- [The basics](#): authentication state, scopes, and endpoint access control.
- [Profile photos](#): upload, display, and default profile images.
- [Setup](#): configure the authentication services these callbacks hook into.
- [Creating an OAuth2-based identity provider](#): implement `mergeAuthUsers` so a custom provider takes part in a merge.